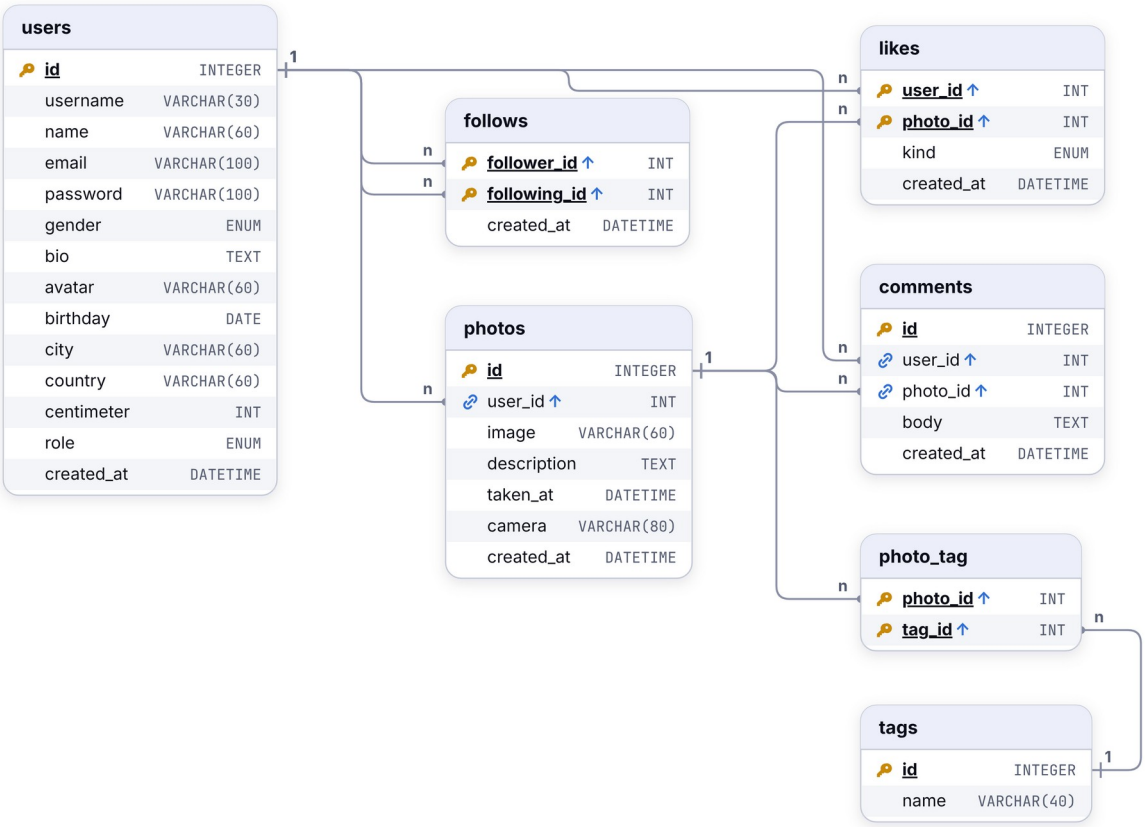
INFORMATIK · QUALIFIKATIONSPHASE

DATENBANK SYSTEME

Relationale Datenbanken und SQL – mit YourSozialNet



Sönke Birkner
soenke.birkner@mps-ki.de

I. Begriffe rund um die Datenwelt

1. Grundbegriffe

Eine Datenbank ist eine Sammlung von Daten, die von einem **Datenbankmanagementsystem (DBMS)** verwaltet wird. Der Ort, an dem die eigentlichen Daten gesammelt werden, wird als **Datenbank** bezeichnet. Eine solche Datenbank kann zum Beispiel aus einzelnen Tabellen bestehen. In welcher Form die Daten in der Datenbank abgelegt werden, muss auch definiert und beschrieben werden. Es handelt sich um die **Metadaten**. Die Metadaten sind also die Datenbank zur Datenbank.

Auf die Datenbank selbst wird durch die Anwendungsprogramme zugegriffen. Das geschieht ausschließlich über das Datenbankmanagementsystem (DBMS). Das DBMS stellt die Schnittstelle zwischen den Anwendungsprogrammen und der Datenbank dar. Auf Seiten der Datenbank erfolgt der Zugriff über standardisierte Schnittstellen, zum Beispiel mit Hilfe der Abfragesprache **SQL** (**Structured Query Language**). Die Anwendungsprogramme senden ihrerseits ihre Anfragen an das DBMS, und diese werden dann an die Datenbank weitergeleitet. So entsteht eine maximale Entkopplung von Anwendungssoftware und Datenbank, d. h. man kann sowohl die Datenbank oder auch die Anwendungssoftware verändern, aktualisieren oder sogar austauschen. Das Gesamtsystem ist das **Datenbanksystem**.

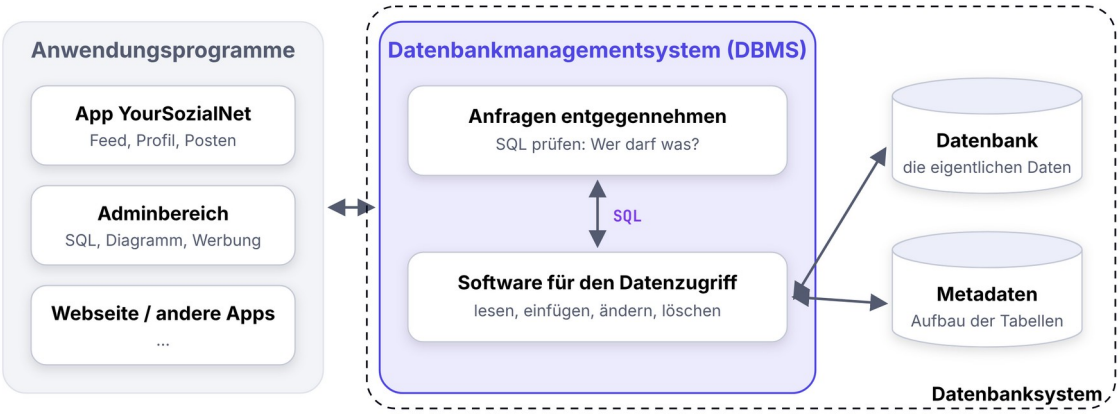


Abb. 1: Komponenten eines Datenbanksystems – am Beispiel von YourSozialNet

Bei YourSozialNet ist das genauso: Die App (Feed, Profil, Posten) und der Adminbereich (SQL) sind zwei Anwendungsprogramme. Beide schicken ihre Anfragen an dasselbe DBMS – und landen in derselben Datenbank.

2. Datenbanken mit mehreren Tabellen

Relationale Datenbanken bestehen in der Regel nicht nur aus einer einzelnen Tabelle, sondern aus mehreren Tabellen. Wie diese Tabellen aufgebaut sind, stellen wir – wie in Java – in einem **Klassendiagramm** dar. So sieht es auch im Programm aus (YourSozialNet: Adminbereich → Diagramm): Jede **Klasse** (Tabelle) ist ein Kasten. Oben steht ihr Name, darunter ihre **Attribute** (Spalten) mit Datentyp.

Als Beispiel dient in diesem Kapitel unsere Schule mit ihren Klassen, Schülerinnen und Schülern und Kursen.

2.1 Primärschlüssel (Primary Key)

Da Zeilen sich stark ähneln können (zwei SchülerInnen könnten den gleichen Namen haben, in demselben Haus wohnen und in die gleiche Klasse gehen), gibt es Attribute, die ein Objekt eindeutig identifizieren sollen. Ohne einen solchen Primärschlüssel könnte der Datensatz einer Tabelle nicht eindeutig angesprochen werden. Der oder die Primärschlüssel einer Klasse werden im Klassendiagramm unterstrichen und haben einen Schlüssel 🔑 davor. Es gibt verschiedene Arten von Primärschlüsseln:

- **Eindeutiger Primärschlüssel:** Es wird ein eindeutiges Attribut einer Tabelle als Schlüssel verwendet, *z. B. die Bezeichnung einer Klasse (es gibt genau eine 5b), E-Mail-Adressen bei Accounts oder die Personalausweisnummer.*
- **Zusammengesetzter Primärschlüssel:** Mehrere Attribute einer Tabelle werden kombiniert und als Schlüssel definiert, *z. B. Nachname, Ort und Geburtstag.* Hierbei muss sichergestellt werden, dass jede Kombination nur einmal vorhanden ist.
- **Künstlicher Primärschlüssel:** Gibt es in einer Tabelle keine eindeutigen Schlüssel, so kann man einen künstlichen Schlüssel definieren. Hierzu wird meist eine fortlaufende Nummer als ID verwendet.

2.2 Fremdschlüssel (Foreign Key)

Ein Fremdschlüssel ist ein Attribut einer Tabelle, das auf einen Primärschlüssel einer anderen Tabelle verweist. Er dient dazu, Verbindungen zwischen verschiedenen Tabellen herstellen zu können. *Ein Schüler hat beispielsweise das Attribut „klasse“. Dies ist der Fremdschlüssel, der den Primärschlüssel der Tabelle klasse enthält.* Im Klassendiagramm hat ein Fremdschlüssel ein ↑ hinter dem Namen (und eine Kette davor).

Oft werden bei einer SQL-Anfrage Daten gewünscht, die aus verschiedenen Tabellen stammen. Über Fremd- und Primärschlüssel können die Tabellen verknüpft werden. Solche Verknüpfungen nennt man **JOINS** (mehr dazu im Abschnitt II.2).

2.3 1-zu-n-Kardinalität

Jedes Objekt einer Klasse steht mit beliebig vielen Objekten der anderen Klasse in Beziehung. In der Gegenrichtung steht jedes Objekt nur mit genau einem Objekt der anderen Klasse in Verbindung. *In eine Klasse (Schule) gehen mehrere Schülerinnen und Schüler, jedoch geht jede Schülerin und jeder Schüler in genau eine Klasse.*

Im Klassendiagramm verbindet eine **Linie** Fremdschlüssel und Primärschlüssel. An den Enden steht die **Kardinalität**: **1** auf der Seite des Primärschlüssels, **n** auf der Seite des Fremdschlüssels.

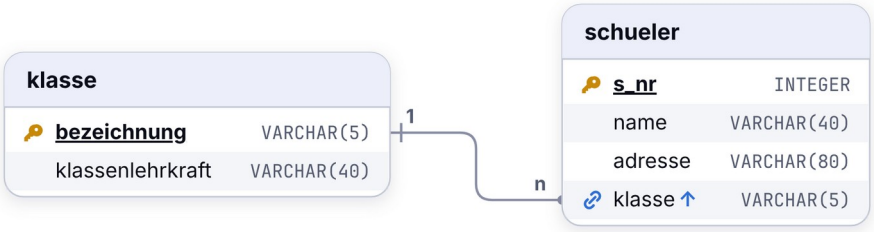


Abb. 2: 1-zu-n – der Fremdschlüssel klasse ↑ steht auf der n-Seite

In den Tabellen sieht das so aus: Der Fremdschlüssel **klasse** in schueler enthält jeweils einen Wert, den es in klasse als Primärschlüssel **bezeichnung** gibt.

KLASSE

bezeichnung 🔑	klassenlehrkraft
5b	Frau Jansen
7a	Herr Okafor
10c	Frau Graf

SCHUELER

s_nr 🔑	name	adresse	klasse ↑
1	Mila	Ostring 4	7a
2	Jonas	Hafenweg 1	5b
4	Ida	Ostring 4	7a
5	Tarek	Am Deich 9	10c

2.4 n-zu-m-Kardinalität

Jedes Objekt einer Klasse steht mit beliebig vielen Objekten der anderen Klasse in Beziehung. In der Gegenrichtung gilt das Gleiche. *In einen Kurs gehen mehrere Schülerinnen und Schüler, und eine Schülerin oder ein Schüler geht in mehrere Kurse.*

Diese Beziehung lässt sich nicht durch einen Fremdschlüssel in einer der beiden Klassen darstellen (ein Attribut kann nicht „Ma1, If3, De2“ enthalten). Dazu wird eine **Hilfstabelle** benötigt, welche die Fremdschlüssel der Klassen verknüpft. Aus einer n-zu-m-Beziehung werden so zwei 1-zu-n-Beziehungen:

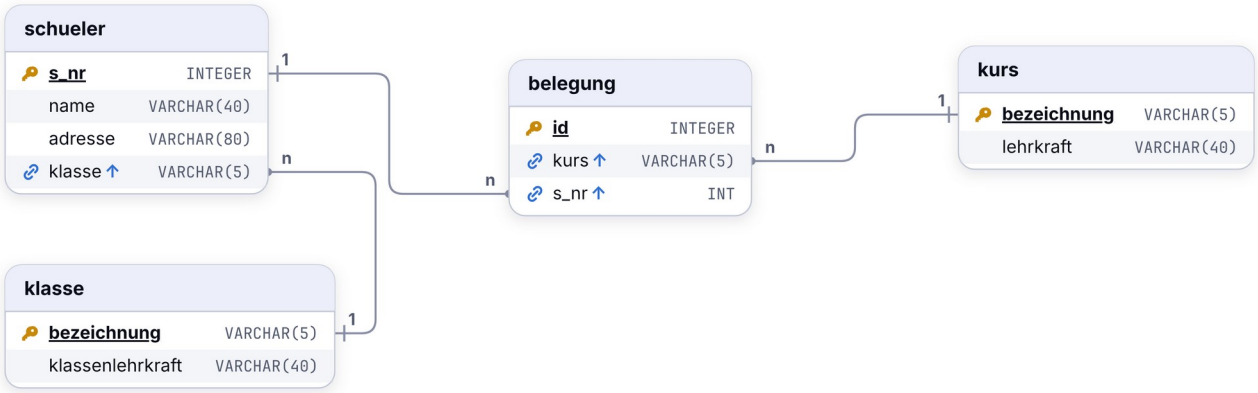


Abb. 3: Die Hilfstabelle belegung verbindet schueler und kurs

id 🔑	kurs ↑	s_nr ↑
1	Ma1	1
2	Ma1	2
3	If3	1
4	If3	4
5	If3	5

An diese Tabelle können weitere Attribute ergänzt werden, falls man diesen Zusammenhang spezifizieren will (*z. B. die Note im Kurs*). Diese Tabelle gilt nicht als neue Klasse, sondern nur als Hilfstabelle, damit der Zusammenhang der n-zu-m-Kardinalität in der Datenbank umgesetzt werden kann.

II. SQL (Structured Query Language)

SQL ist die Datenbanksprache. Mit dieser können Datenbanken erstellt und verändert werden. Außerdem können Abfragen vorgenommen werden. Jeder Zugriff aus der Anwender-Software wird in einen SQL-Befehl umgewandelt – auch jedes Like, das du in einer App setzt.

1. Datenbankabfragen mit SQL

In einer relationalen Datenbank werden Informationen in Tabellen (**Klassen**) repräsentiert. Die Zeilen dieser Tabelle heißen **Objekte**, die Spalten **Attribute**. Am Anfang hat dein Netzwerk nur eine einzige Klasse: **users**.

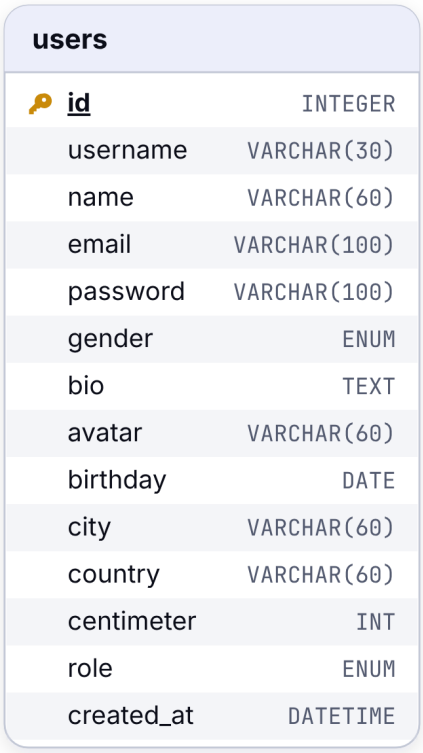


Abb. 4: Die Klasse users im Diagramm von YourSozialNet

So sehen einige Objekte der Klasse users aus:

Klasse (Tabelle): users

id	username	name	gender	birthday	city	centimeter
2	noobqueen	Mia Sander	female	2009-09-16	Kiel	165
4	levi.kicks	Levi Schumacher	male	2009-04-11	**Kiel**	184
6	coach_ralf	Ralf Opitz	male	1980-07-10	Kiel	179
8	alina.daily	Alina Keller	female	2005-08-22	Hamburg	170
14	inkandmila	Mila Hoffmann	female	2010-04-21	Leipzig	160
37	levi.berlin	Levi Schumacher	male	1992-05-03	Berlin	181

← **Objekt** = eine Zeile, ein Datensatz

🔑 **Primärschlüssel** id: jede Zeile hat eine andere **Attribut** = eine Spalte, z. B. city

Abb. 5: Ausschnitt aus der Tabelle users

Damit in der Datenbank nicht händisch nach Informationen gesucht werden muss, können mit SQL-Befehlen die Inhalte der Datenbank gezielt ausgewählt oder sortiert werden. Das Ergebnis einer SQL-Abfrage ist immer wieder eine Tabelle, welche sich aus der Abfrage ergibt.



Abb. 6: Eine SQL-Abfrage wählt Spalten und Zeilen aus

Dabei folgen SQL-Befehle immer einer bestimmten Grundstruktur:

- Der **SELECT**-Befehl gibt an, welche Attribute ausgegeben werden sollen. Mit einem ***** werden alle Attribute einer Klasse ausgegeben.
- Im **FROM**-Befehl wird die Klasse angegeben, welche betrachtet wird.
- Im **WHERE**-Befehl können noch weitere Bedingungen gesetzt werden.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Bedingung>
```

BEISPIEL · PERSONEN, DIE KLEINER ALS 1,60 M SIND

```
SELECT id, name, centimeter
FROM users
WHERE centimeter < 160
```

1.1 Projektion (Ausdrücke und Auswahl von Attributen)

Bei einer Projektion wird eine Klasse gezielt auf bestimmte Spalten reduziert. Es können auch Spalten neu gebildet werden, wenn Inhalte zusammengefasst oder umgerechnet werden sollen. Diese zusätzlichen Spalten werden mit dem Keyword **AS** benannt. Ausdrücke können Attribute, konstante Werte, arithmetische Operationen (+, -, *, /) oder Funktionen (`NOW()`) sein.

AUFBAU

```
SELECT <Liste von Attributen>,
       <Ausdruck> AS <Name>
FROM <Klasse>
```

BEISPIEL · GRÖSSE IN METER UMGERECHNET

```
SELECT id, name,
       centimeter / 100 AS meter
FROM users
```

Funktion	Bedeutung
`NOW()`	liefert das aktuelle Datum mit Uhrzeit
`DATEDIFF(<datum1>, <datum2>)`	liefert die Anzahl der Tage, die zwischen den beiden Daten liegen
`FLOOR(<x>)`	rundet `<x>` ab auf eine ganze Zahl
`YEAR(<datum>)`	liefert das Jahr eines Datums (genauso `MONTH`, `DAY`, `HOUR`)

Aufgabe 1

Notiere für die folgenden Abfragen einen SQL-Befehl. Deine Ausgabe sollte eine geeignete Auswahl an Attributen haben. Halte diesen digital oder analog fest!

a) Gib alle Einträge aus der Tabelle users aus.

b) Gib alle Benutzernamen (username) und echten Namen (name) aller Einträge aus users aus.

c) Gib die Wohnorte aller Mitglieder aus.

d) Gib die Größe der User in Fuß (feet) an (1 cm = 0,0328 ft).

e) Berechne das Alter der User und zeige den Namen und das Alter an. Nutze die Funktionen von der vorherigen Seite.

f) Überprüfe mit deiner Lösung das Alter von Noah (Username noah.sz). Stimmt dein Ergebnis? Falls nicht, überlege dir, woran das liegen könnte.

1.2 Sortierung und Doppelungen

DISTINCT kann nach dem **SELECT**-Befehl eingefügt werden. Dieser verhindert Doppelungen in der Ausgabe. Es werden identische Zeilen zusammengefügt. Dies ist praktisch, wenn nur bestimmte Attribute betrachtet werden. Beispielsweise können so alle Länder ausgegeben werden.

AUFBAU

```
SELECT DISTINCT <Liste von Attributen>
FROM <Klasse>
```

BEISPIEL · HERKUNFTSLÄNDER DER USER

```
SELECT DISTINCT country
FROM users
```

ORDER BY – Um Ergebnisse nach einem Attribut zu sortieren, kann der Befehl **ORDER BY <Attribut>** angefügt werden. Dieser sortiert die Liste alphabetisch bzw. nach Größe. **ASC** sorgt für eine ansteigende und **DESC** für eine absteigende Sortierung.

LIMIT – Mit **LIMIT <Anzahl>** wird die Ausgabe auf die ersten Zeilen begrenzt. Zusammen mit ORDER BY erhältst du so z. B. die „Top 3“.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
ORDER BY <Attribut> ASC/DESC
LIMIT <Anzahl>
```

BEISPIEL · DIE DREI GRÖSSTEN USER

```
SELECT name, centimeter
FROM users
ORDER BY centimeter DESC
LIMIT 3
```

Aufgabe 2

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Gib alle Wohnorte in einer sortierten Liste aus, bei der jede Stadt einmalig auftritt.

b) Ordne die Mitglieder nach ihrer Größe absteigend.

c) Kehre die Reihenfolge der Tabelle **users** um.

d) Gib die fünf zuletzt registrierten User aus.

1.3 Selektion (Auswahl von Objekten)

Bei der Selektion im WHERE-Befehl können Objekte (Zeilen) einer Klasse mithilfe von Bedingungen ausgewählt werden.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Bedingung>
```

BEISPIEL · ALLE USER AUS KIEL

```
SELECT *
FROM users
WHERE city = 'Kiel'
```

Vergleichsoperationen – In SQL wird die Gleichheit mit „=“ und nicht mit „==“ überprüft. Texte stehen in einfachen Anführungszeichen `'Kiel'`, Zahlen ohne. Sonstige Operationen sind analog zu Java.

Gleich	Ungleich	Größer	Kleiner	Größergleich	Kleinergleich
`=`	`!=`	`>`	`<`	`>=`	`<=`

Ähnlichkeit (LIKE) – Mit dem Schlüsselwort LIKE kann man Objekte auswählen, deren Attribute einer vorgegebenen Zeichenkette ähnlich sind. Das Zeichen `_` dient als Platzhalter für ein beliebiges Zeichen und `%` als Platzhalter für eine Zeichenkette beliebiger Länge. Die Bedingung `username LIKE 'levi%'` erfüllen beispielsweise Objekte mit dem Username „levi.kicks“ oder „levi_fan_09“, jedoch nicht „mama_von_levi“. LIKE unterscheidet dabei nicht zwischen Groß- und Kleinschreibung.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Attribut> LIKE <Muster>
```

BEISPIEL · ALLE USER, DEREN USERNAME MIT „LEVI“ BEGINNT

```
SELECT username, name
FROM users
WHERE username LIKE 'levi%'
```

Logische Operationen (AND, OR, NOT) – Auch die logischen Operationen gibt es in SQL. Diese können den **WHERE**-Befehl erweitern.

ALLE USER, DIE AUS BERLIN KOMMEN UND FELIX HEISSEN

```
SELECT username, name, city
FROM users
WHERE city = 'Berlin' AND name LIKE 'Felix%'
```

ALLE USER, DIE AUS BERLIN KOMMEN ODER FELIX HEISSEN

```
SELECT username, name, city
FROM users
WHERE city = 'Berlin' OR name LIKE 'Felix%'
```

ALLE USER, DIE AUS BERLIN KOMMEN UND NICHT FELIX HEISSEN

```
SELECT username, name, city
FROM users
WHERE city = 'Berlin' AND name NOT LIKE 'Felix%'
```

Hinweis: `AND name NOT LIKE` lässt sich in SQL auch mit `AND NOT name LIKE` schreiben.

Leere Werte (IS NULL) – Hat ein User etwas nicht angegeben, steht dort **NULL** („nichts“). NULL ist kein Wert: `city = NULL` findet deshalb nie etwas. Stattdessen schreibt man `city IS NULL` bzw. `city IS NOT NULL`.

Wechselwirkungen von Einschränkungen – AND bindet stärker als OR (wie Punkt- vor Strichrechnung). Um Bedingungen logisch zu kapseln, können – wie in der Mathematik – Klammern genutzt werden.

Alle User aus Kiel, egal ob männlich oder weiblich – nur eine Möglichkeit funktioniert

MÖGLICHKEIT 1

```
SELECT name, city, gender
FROM users
WHERE city = 'Kiel' AND
      gender = 'male' OR
      gender = 'female'
```

MÖGLICHKEIT 2

```
SELECT name, city, gender
FROM users
WHERE city = 'Kiel' AND
      (gender = 'male' OR
       gender = 'female')
```

Aufgabe 3

Notiere für die folgenden Abfragen (ab Aufgabe b) einen SQL-Befehl.

a) Teste beide Möglichkeiten vom letzten Abschnitt und finde heraus, worin der Unterschied der Ausgaben liegt. Welche Möglichkeit passt zur Aufgabe?

b) Gib alle User aus, die nicht aus Deutschland kommen. Schau dir dein Ergebnis genau an: Stimmt es wirklich?

c) Zeige nur Levi Schumacher an. Wie viele Treffer bekommst du? Wie kannst du sicherstellen, dass du genau einen bestimmten Levi bekommst?

d) Prüfe, ob es minderjährige User gibt.

e) ★ Finde zuerst heraus, wann levi.kicks geboren ist. Gib dann alle User aus, die im selben Jahr geboren sind, und ordne sie nach Geburtsdatum.

f) Erstelle eine Liste mit Usern, deren E-Mail-Adresse zu deinem Netzwerk gehört (sie endet auf *@name-deines-netzwerks.test*).

g) Gib eine Liste aller männlichen User an, die größer als 1,85 m sind. Ordne die Liste nach Wohnort.

h) Prüfe, ob es einen User gibt, der Felix heißt und nicht aus Berlin kommt.

i) ★ Die Bundeswehr sucht Rekrutinnen und Rekruten. Gib die Namen und das Alter aller Männer über 165 cm und aller Frauen über 160 cm aus (es werden keine Panzerfahrerinnen und Panzerfahrer gesucht).

j) Erna Lüders sucht ihren Enkel. Sein Vorname war entweder Lasse oder Lars. Er kommt wohl aus Berlin. Kannst du herausfinden, ob ihr Enkel in deinem Netzwerk ist?

★ zum Knobeln

1.4 Gruppierungen und Aggregatfunktionen

In SQL können Zeilen nach bestimmten Kriterien zusammengefasst werden.

Befehl	Bedeutung
`MIN(<Attribut>)` / `MAX(<Attribut>)`	kleinster bzw. größter Wert eines Attributs
`SUM(<Attribut>)` / `AVG(<Attribut>)`	Summe bzw. Durchschnitt der Werte eines Attributs
`COUNT(*)` / `COUNT(<Attribut>)`	zählt die Zeilen bzw. die Zeilen, in denen das Attribut nicht NULL ist
`GROUP BY <Attribut>`	fasst alle Zeilen mit gleichem Wert zu einer Gruppe zusammen; die Funktionen oben rechnen dann je Gruppe

GRÖSSE DES GRÖSSTEN USERS

```
SELECT MAX(centimeter) AS maximale_groesse
FROM users
```

LISTE DER STÄDTE UND DER JEWEILIGEN USER-ANZAHL

```
SELECT city, COUNT(*) AS anzahl
FROM users
GROUP BY city
ORDER BY anzahl DESC
```

Aufgabe 4

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Gib an, wann sich zuletzt ein Mitglied registriert hat.

b) Gib das Alter des ältesten Mitglieds an.

c) Betrachte die beiden Befehle und überlege, was diese ausgeben sollen. Teste deine Vermutung. Zu welchem Wert gehört der Name im zweiten Befehl? Erkläre, warum das Ergebnis anders ist als erwartet.

BEFEHL 1

```
SELECT name, MAX(centimeter)
FROM users
```

BEFEHL 2

```
SELECT name, MAX(centimeter),
       MIN(centimeter)
FROM users
```

d) Ermittle, wie viele männliche und wie viele weibliche User dein Netzwerk hat. Was steht außerdem in der Ausgabe?

e) ★ Prüfe, ob es User gibt, die am gleichen Tag Geburtstag haben. Wer ist es?

f) Gib die Namen der drei Städte mit den meisten Usern an.

g) Vergleiche die beiden Befehle. Warum kommen verschiedene Zahlen heraus? Finde die User, die im zweiten Befehl fehlen.

BEFEHL 1

```
SELECT COUNT(*)
FROM users
```

BEFEHL 2

```
SELECT COUNT(centimeter)
FROM users
```

h) Wie viele User kommen aus Hamburg? Vergleiche `city = 'Hamburg'` mit `city LIKE 'Hamburg %'`. Hast du wirklich alle Hamburger gefunden?

★ zum Knobeln

2. Photos und Comments

YourSozialNet wäre kein echtes soziales Netzwerk, wenn es keine Bilder geben würde. Daher erweitern wir unser Netzwerk nun um zwei weitere Klassen: **photos** und **comments**.

- **photos** ist der Klassiker einer 1-zu-n-Beziehung: Ein User kann mehrere Fotos posten, jedes Foto gehört aber genau einem User. Der Fremdschlüssel `user_id ↑` steht deshalb in photos.
- **comments** hat gleich **zwei** 1-zu-n-Beziehungen: Ein User schreibt viele Kommentare (`user_id ↑`) und ein Foto hat viele Kommentare (`photo_id ↑`). Jeder Kommentar gehört zu genau einem Verfasser und genau einem Foto.

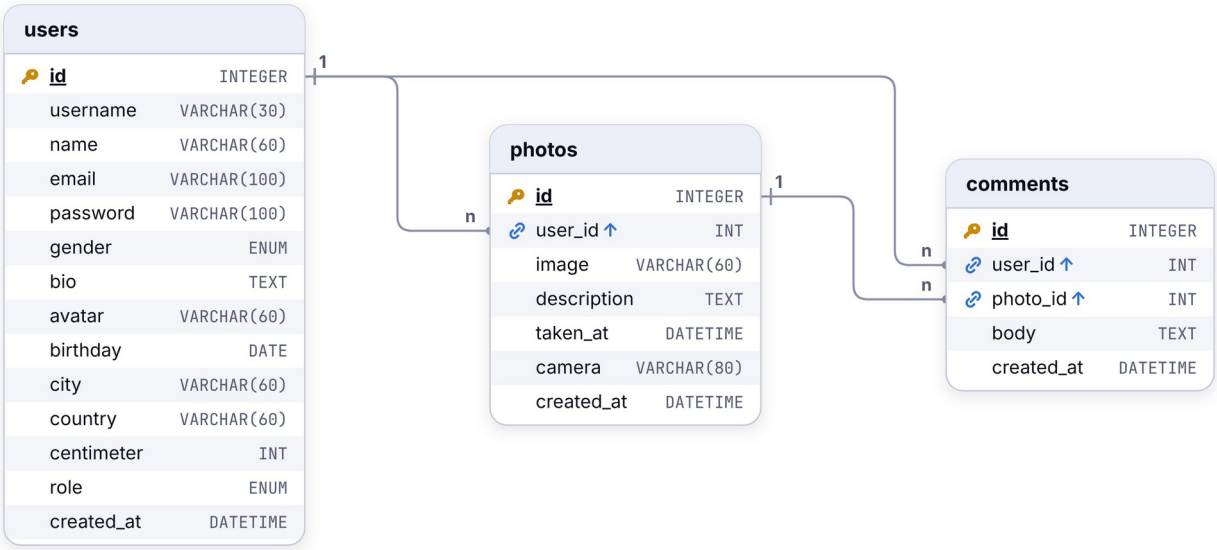


Abb. 7: users, photos und comments

JOIN

Es gibt zahlreiche Möglichkeiten, in SQL auf zwei Tabellen zuzugreifen. Meistens werden dazu JOIN-Befehle genutzt. Wir beschränken uns zuerst auf den (INNER) JOIN. Nach **ON** steht, welcher Fremdschlüssel zu welchem Primärschlüssel gehört. Da Attributnamen zum Teil in beiden Klassen auftauchen (z. B. `id`), können bzw. müssen diese eindeutig mit der Punktnotation (wie in Java) angegeben werden: `photos.user_id`, `users.id`.



Abb. 8: Der JOIN hängt an jedes Foto die Zeile seines Users

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse1>
JOIN <Klasse2>
  ON <Fremdschlüssel> =
<Primärschlüssel>
WHERE <Bedingung>
```

BEISPIEL · BILDER UND IHRE BESITZER

```
SELECT username, image, description
FROM users
JOIN photos
  ON photos.user_id = users.id
ORDER BY username
```

Aufgabe 5

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Gib alle Fotos zusammen mit dem Benutzernamen des Besitzers aus.

b) Gib eine absteigend sortierte Tabelle aus, in der die User und die Anzahl ihrer geposteten Fotos stehen.

c) Gib alle Fotos von Levi Schumacher aus Kiel mit ihrer Beschreibung aus.

d) Gib alle Kommentare und die Benutzernamen der Verfasser aus München aus.

e) Gib alle Kommentare aus, die unter dem Foto mit der Ostsee (`dummy/linus_seascape.webp`) stehen.

f) Welches Foto hat die meisten Kommentare?

g) Ein Kommentar taucht auffällig oft auf. Finde ihn und seinen Verfasser.

h) Die Moderation deines Netzwerks bittet um Mithilfe: Einige User verbreiten, andere seien „Echsenmenschen von der Rückseite des Mondes“. Wer hat solche Kommentare geschrieben? Gib E-Mail, Benutzername und Kommentar aus. Prüfe dein Ergebnis: Sind wirklich alle Treffer Verschwörungstheoretiker?

3. Tags, Follows und Likes

Natürlich gibt es noch ein paar weitere Funktionen in YourSozialNet, die du auch von anderen Netzwerken kennst. Alle drei sind **n-zu-m-Kardinalitäten**, weswegen – wie bei den Kursen in Kapitel I – Hilfstabellen benötigt werden.

- **tags**: Ein Foto kann viele Tags haben (#kiel, #segeln), und ein Tag steht an vielen Fotos. Die Klasse **tags** speichert jeden Tag nur einmal, die Hilfstabelle **photo_tag** verknüpft Fotos und Tags. Das ist die „klassische“ n-zu-m-Beziehung.
- **follows**: Ein User folgt vielen Usern und hat viele Follower. Hier verbindet die Hilfstabelle die Klasse users **mit sich selbst**: `follower_id ↑` (wer folgt) und `following_id ↑` (wem gefolgt wird) zeigen beide auf users.
- **likes**: Ein User liked viele Fotos, ein Foto wird von vielen Usern gelikt. Die Hilfstabelle bekommt ein zusätzliches Attribut **kind** – die Art des Likes (heart, thumb, laugh).

Bei den drei Hilfstabellen ist der Primärschlüssel **zusammengesetzt** (siehe I.2.1): `(photo_id, tag_id)`, `(follower_id, following_id)` und `(user_id, photo_id)`. So kann niemand ein Foto zweimal liken.

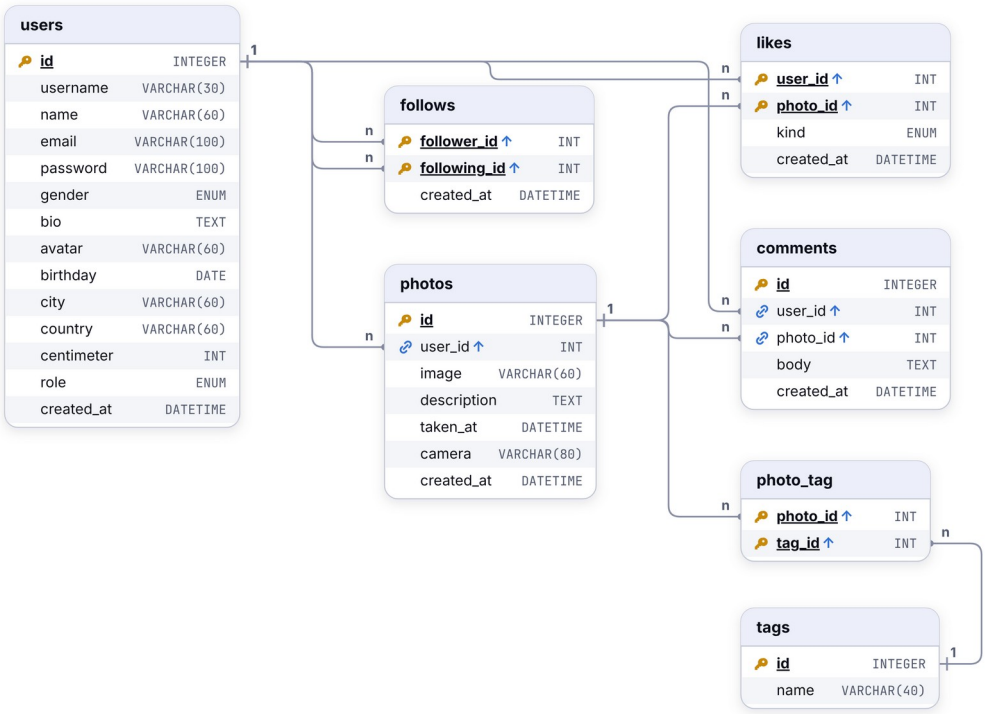


Abb. 9: Das Netzwerk mit Tags, Follows und Likes

JOIN über eine Hilfstabelle

Um eine n-zu-m-Beziehung auszugeben, geht man über die Hilfstabelle: von der einen Klasse zur Hilfstabelle und von dort zur anderen Klasse. Es werden also drei Tabellen verbunden.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse1>
JOIN <Hilfstabelle>
  ON <Fremdschlüssel1> =
<Primärschlüssel1>
JOIN <Klasse2>
  ON <Primärschlüssel2> =
<Fremdschlüssel2>
WHERE <Bedingung>
```

BEISPIEL · ALLE TAGS DES FOTOS MIT DER OSTSEE (SEASCAPE)

```
SELECT photos.image, tags.name
FROM photos
JOIN photo_tag ON photo_tag.photo_id = photos.id
JOIN tags      ON tags.id = photo_tag.tag_id
WHERE photos.image = 'dummy/linus_seascape.webp'
```

Übrigens: comments funktioniert genauso. Ein Kommentar verbindet einen User mit einem Foto – wer von users über comments zu photos will, braucht also auch zwei JOINs.

KOMMENTARE MIT VERFASSER UND FOTO

```
SELECT users.username, comments.body, photos.image
FROM users
JOIN comments ON comments.user_id = users.id
JOIN photos   ON comments.photo_id = photos.id
```

ALLE USER, DIE DAS FOTO MIT DER OSTSEE GELIKT HABEN – UND WIE

```
SELECT users.username, likes.kind
FROM users
JOIN likes  ON likes.user_id = users.id
JOIN photos ON likes.photo_id = photos.id
WHERE photos.image = 'dummy/linus_seascape.webp'
```

Eine Klasse zweimal: Aliasse

Bei follows braucht man die Klasse users **zweimal**: einmal für den, der folgt, und einmal für den, dem gefolgt wird. Damit SQL die beiden auseinanderhalten kann, bekommt jede einen eigenen Namen, einen **Alias**: `users AS fan` und `users AS star`.

WEM FOLGT LEVI.KICKS?

```
SELECT fan.username AS folgt, star.username AS wem
FROM follows
JOIN users AS fan  ON follows.follower_id  = fan.id
JOIN users AS star ON follows.following_id = star.id
WHERE fan.username = 'levi.kicks'
```

Aufgabe 6

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Unter welchen Bildern stehen die Echsen-Kommentare aus Aufgabe 5 h)? Gib Benutzername, Kommentar und Bild aus.

b) ★ Welches Foto hat die meisten Kommentare – und wem gehört es? *Achtung: Hier gibt es zwei verschiedene User – den Verfasser des Kommentars und den Besitzer des Fotos.*

c) Gib eine Tabelle aus, in der alle Fotos zusammen mit ihren Tags stehen.

d) Wie viele Bilder haben den Tag „abenteuerlust“? Vergleiche dein Ergebnis mit `LIKE '%abenteuer%'`. Welche Zahl ist richtig?

e) Welche Tags hat Alina Keller genutzt?

f) Gib die 10 beliebtesten Fotos aus (die meisten Likes) – mit dem Benutzernamen des Besitzers.

g) Erinnerst du dich an Erna aus Aufgabe 3 j)? Wem folgt sie? Ist ihr Enkel dabei?

h) ★ Welche User folgen sich gegenseitig?

★ zum Knobeln

4. Tiefer in SQL

Mit SELECT, WHERE, GROUP BY und JOIN kannst du schon sehr viel herausfinden. Manche Fragen brauchen aber mehr: eine Menge von Werten, ein Ergebnis, mit dem man weiterrechnet, oder Zeilen, die es auf der anderen Seite gar nicht gibt.

4.1 IN

Der IN-Befehl wird in der WHERE-Klausel verwendet, um zu prüfen, ob ein Wert zu einer bestimmten Menge von Werten gehört. Er ist eine übersichtliche Alternative zu mehreren OR-Bedingungen. **NOT IN** prüft das Gegenteil.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Attribut> IN (<Menge von Werten>)
```

BEISPIEL · ALLE USER, DIE IN DER SCHWEIZ ODER IN ÖSTERREICH WOHNEN

```
SELECT username, country
FROM users
WHERE country IN ('Schweiz', 'Österreich')
```

4.2 Geschachtelte Abfragen (Kapselung)

Das Ergebnis einer SQL-Abfrage ist immer eine Tabelle. Deshalb kann man eine Abfrage in eine andere hineinschreiben – in Klammern. Man nennt das eine **Unterabfrage** (oder Kapselung). Die **innere** Abfrage wird zuerst ausgeführt, mit ihrem Ergebnis rechnet dann die **äußere** Abfrage weiter.

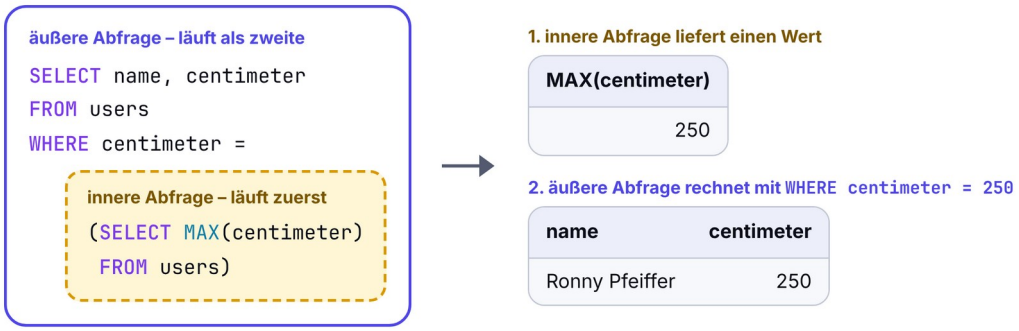


Abb. 10: Die innere Abfrage liefert den Wert, mit dem die äußere rechnet

Eine Unterabfrage kann an drei Stellen stehen. Je nachdem, wo sie steht, muss sie etwas anderes liefern:

Wo?	Was muss die Unterabfrage liefern?	Beispiel
nach `=`, `<`, `>` …	genau **einen Wert** (eine Zeile, eine Spalte)	`WHERE centimeter > (SELECT AVG(centimeter) FROM users)`
nach `IN` / `NOT IN`	eine **Liste** (eine Spalte, beliebig viele Zeilen)	`WHERE id IN (SELECT user_id FROM photos)`
nach `FROM`	eine ganze **Tabelle** – sie braucht einen Namen mit AS	`FROM (SELECT …) AS t`

EIN WERT: ALLE USER, DIE GRÖSSER ALS DER DURCHSCHNITT SIND

```
SELECT name, centimeter
FROM users
WHERE centimeter > (SELECT AVG(centimeter)
                    FROM users)
```

4.3 HAVING

WHERE filtert einzelne Datensätze **vor** der Gruppierung (GROUP BY) und entscheidet damit, welche Zeilen überhaupt in die Gruppenbildung eingehen. Aggregatfunktionen wie SUM oder COUNT sind hier nicht erlaubt.

HAVING filtert **nach** der Gruppierung und legt fest, welche gebildeten Gruppen im Ergebnis erscheinen. Dabei dürfen Aggregatfunktionen verwendet werden, da sich die Bedingung auf berechnete Gruppenwerte bezieht.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse>
WHERE <Bedingung für Zeilen>
GROUP BY <Attribut>
HAVING <Bedingung für Gruppen>
```

BEISPIEL · ALLE USER AUS KIEL MIT MEHR ALS 5 BILDERN

```
SELECT users.name, COUNT(photos.id) AS fotos
FROM users
JOIN photos ON users.id = photos.user_id
WHERE users.city = 'Kiel'
GROUP BY users.id
HAVING COUNT(photos.id) > 5
```

4.4 JOIN: mehr Varianten

Neben dem „Standard“-JOIN gibt es noch mehr Möglichkeiten, zwei Tabellen zusammenzufügen. Sie werden auch mit einem ON und deren Primär-/Fremdschlüssel verknüpft.

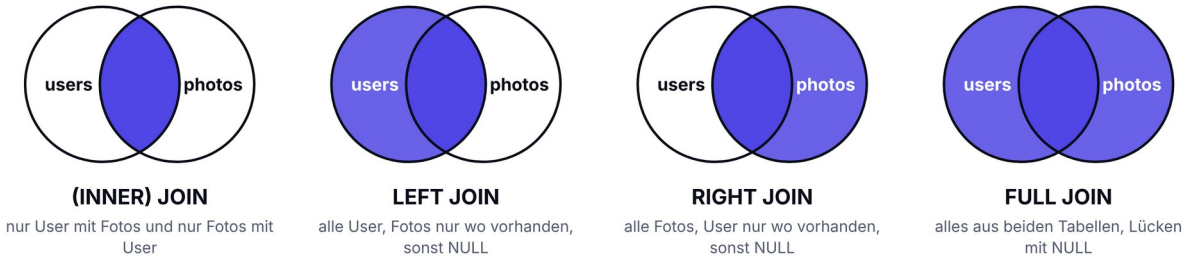


Abb. 11: Welche Zeilen bleiben bei welchem JOIN übrig?

- Der **INNER JOIN** gibt nur die Informationen zurück, die in beiden Tabellen Übereinstimmungen besitzen (INNER JOIN = JOIN).
- Der **LEFT JOIN** gibt alle Informationen der linken Tabelle (Tabelle 1) zurück, sowie die Informationen, die mit der rechten Tabelle (Tabelle 2) übereinstimmen. Gibt es keine Übereinstimmung, wird NULL ausgegeben.
- Der **RIGHT JOIN** gibt alle Informationen der rechten Tabelle (Tabelle 2) zurück, sowie die Informationen, die mit der linken Tabelle (Tabelle 1) übereinstimmen. Gibt es keine Übereinstimmung, wird NULL ausgegeben.
- Der **FULL (OUTER) JOIN** gibt alle Informationen aus beiden Tabellen aus. Sollten Werte, egal aus welcher Tabelle, keine Übereinstimmung haben, wird NULL ausgegeben.

AUFBAU

```
SELECT <Liste von Attributen>
FROM <Klasse1>
[LEFT | RIGHT | FULL] JOIN
<Klasse2>
  ON <Fremdschlüssel> =
<Primärschlüssel>
WHERE <Bedingung>
```

BEISPIEL · ALLE USER MIT IHREN BILDERN – AUCH DIE OHNE BILDER

```
SELECT users.name, photos.image
FROM users
LEFT JOIN photos ON photos.user_id = users.id
ORDER BY users.name
```

Der Trick: Wer **keinen** Partner hat, bekommt NULL. Mit `WHERE photos.id IS NULL` bleiben also genau die User ohne Foto übrig.

Aufgabe 7

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Liste alle User auf, die aus Deutschland kommen – egal, ob sie „Deutschland“, „Germany“, „deutschland“ oder „DE“ angegeben haben.

b) Gib alle User aus, die noch nie ein Foto gepostet haben. Nutze NOT IN.

c) Gib den Namen der größten Person aus – diesmal sicher richtig (vgl. Aufgabe 4 c).

d) Welche Tags wurden mindestens 8-mal verwendet?

e) Finde die 5 Fotos, die von niemandem gelikt wurden. Nutze den LEFT JOIN.

f) ★ Liste die Bilder, die nicht gelikt wurden, noch einmal auf – diesmal mit dem RIGHT JOIN. Was musst du gegenüber Aufgabe 7 e) ändern?

g) Beide Befehle sollen zählen, wie viele Fotos jeder User hat. Bei manchen Usern unterscheiden sich die Zahlen. Bei wem und warum?

BEFEHL 1

```sql
SELECT users.username, COUNT(*)
FROM users
LEFT JOIN photos
  ON photos.user_id = users.id
GROUP BY users.id
```

BEFEHL 2

```sql
SELECT users.username, COUNT(photos.id)
FROM users
LEFT JOIN photos
  ON photos.user_id = users.id
GROUP BY users.id
```

h) Einer der beiden Befehle läuft nicht. Welcher, und warum? Was musst du ändern?

BEFEHL 1

```sql
SELECT city, COUNT(*)
FROM users
WHERE COUNT(*) > 4
GROUP BY city
```

BEFEHL 2

```sql
SELECT city, COUNT(*)
FROM users
GROUP BY city
HAVING COUNT(*) > 4
```

i) ★ Welche User haben mehr Fotos gepostet als der Durchschnitt aller User, die posten?

★ zum Knobeln

5. Alles zusammen: 10 Aufgaben zum Üben

Hier kommt alles vor, was du bisher gelernt hast: Projektion, Selektion, Sortierung, Gruppierung, JOIN und Unterabfragen. Überlege bei jeder Aufgabe zuerst: *Welche Tabellen brauche ich?* Schau dazu ins Diagramm.

Aufgabe 8

Notiere für die folgenden Abfragen einen SQL-Befehl.

a) Gib eine Liste aus, in der alle User (Username, Name, Country) sortiert nach ihrem Namen stehen.

b) Erstelle eine Liste, in der steht, wie viele User aus den jeweiligen Städten kommen – sortiert nach der Anzahl der User.

c) Wie viele User haben eine E-Mail-Adresse deines Netzwerks und wie viele eine externe?

d) Gib die drei jüngsten User aus.

e) Welcher User hat die meisten Fotos gepostet?

f) * Gib eine Liste aus, in der für jeden User die Anzahl der von ihm verteilten Likes steht – auch für User ohne Like.

g) Welche Fotos hat hugo.ranked kommentiert, und was hat er geschrieben?

h) Gib eine Liste aus, in der alle Bildbeschreibungen stehen, die von Laura Mauer oder Marlo Eberhart gelikt wurden.

i) ★ Welcher Tag wird von den meisten verschiedenen Usern benutzt?

j) * Welche User haben noch nie etwas gepostet, gelikt oder kommentiert?

* nur mit tieferen SQL-Befehlen lösbar (Abschnitt 4) · ★ zum Knobeln

6. Datenbankinhalte ändern

SQL verwaltet nicht nur die Abfragen einer Datenbank. Die gesamte Datenbank wird durch SQL erstellt und verändert. Dazu gibt es weitere Befehle, die analog zum SELECT-Befehl genutzt werden können: Mit **INSERT** werden neue Daten in die Datenbank eingefügt, mit **UPDATE** können vorhandene Daten verändert werden und **DELETE** löscht Daten.

6.1 Einfügen

Das Einfügen von Daten geschieht über **INSERT**. Dabei müssen die entsprechende Tabelle und deren Attribute nach **INTO** angegeben werden. Die Liste der Attribute muss nicht vollständig sein, ausgelassene Attribute werden mit NULL oder ihrem Standardwert (*DEFAULT*) gefüllt. **VALUES** leitet dann die Werte für das neue Objekt ein. Es können mehrere Objekte gleichzeitig eingefügt werden.

AUFBAU

```
INSERT INTO <Klasse>
  (<Liste der Attribute>)
VALUES
  (<Werte vom 1. Objekt>),
  (<Werte vom 2. Objekt>);
```

BEISPIEL · ERSTELLE DEN NEUEN USER GÜNTHER

```
INSERT INTO users
  (username, name, email, password,
   gender, birthday, city, country,
   centimeter)
VALUES
  ('guenther37', 'Günther Müller',
   'guenther@mail.test', '123passwort',
   'male', '1987-07-07', 'Kiel',
   'Deutschland', 178);
```

Statt fester Werte kann INSERT auch das Ergebnis einer Abfrage einfügen – wieder eine Kapselung. Dann fällt VALUES weg:

ALLE USER AUS KIEL FOLGEN DEM USER MIT DER ID 71

```
INSERT INTO follows (follower_id, following_id)
SELECT id, 71
FROM users
WHERE city = 'Kiel'
```

Aufgabe 9

Notiere für die folgenden Aufgaben einen SQL-Befehl und führe ihn aus. Kontrolliere das Ergebnis in der App.

a) Füge eine fiktive Person zu deiner Datenbank hinzu.

b) Überprüfe, welche id diese Person bekommen hat, obwohl du sie nicht angegeben hast. Welche Werte stehen in role und created_at? Woher kommen sie? (Schau ins Diagramm.)

c) Melde dich in der App mit dieser Person an. Es klappt nicht! Aber warum? Finde heraus, woran es liegt.

d) Schreibe unter ein Foto deiner Wahl Kommentare im Namen von mindestens drei verschiedenen Usern. Schau in der App nach: Kann man erkennen, dass die Kommentare nicht echt sind?

e) Gönn dir ein paar mehr Follower: Alle User aus München sollen deinem Admin-Konto folgen. Erstelle dafür keine neuen Personen.

f) * Lass 20 User ein Foto deiner Wahl mit 😂 (laugh) liken. Was passiert, wenn einer von ihnen das Foto schon gelikt hat? Lies die Fehlermeldung und erkläre sie mit dem Diagramm.

* nur mit tieferen SQL-Befehlen lösbar (Abschnitt 4)

6.2 Verändern

Zum Verändern von Daten wird der **UPDATE**-Befehl genutzt. Hierbei können einzelne Attribute beliebig neu besetzt werden. Bei welchen Objekten die Daten geändert werden, wird im WHERE-Befehl definiert. **Ohne WHERE werden alle Zeilen geändert!**

AUFBAU

```
UPDATE <Klasse>
SET <Attribut> = <neuer Wert>,
    <Attribut> = <neuer Wert>, …
WHERE <Bedingung>
```

BEISPIEL · ALLE HAMBURGER ZIEHEN NACH KIEL, WEIL BESSER

```
UPDATE users
SET city = 'Kiel'
WHERE city = 'Hamburg'
```

Aufgabe 10

Notiere für die folgenden Aufgaben einen SQL-Befehl und führe ihn aus.

a) Versuche, das Geschlecht aller User aus Dresden auf „keine Angabe“ zu setzen. Lies die Fehlermeldung: Welche Werte sind erlaubt? Wo im Diagramm steht das? Warum ist so eine Einschränkung sinnvoll?

b) Suche dir 10 Bilder von anderen Usern deiner Datenbank aus und mache sie zu deinen Bildern. Wie sieht dein Profil danach aus?

c) * ★ Lenke alle Likes auf eines deiner Bilder um. Warum klappt das nicht? *Tipp: Schau dir den Primärschlüssel von likes an.* Schaffst du es, wenigstens die Likes umzulenken, die möglich sind?

d) Ersetze die Körpergröße aller User durch `FLOOR(RAND() * 50) + 150`. Was erzeugt dieser Befehl? Führe ihn zweimal aus und vergleiche.

e) * User, die ihre Boote fotografiert haben, möchten ihren Wohnort durch „auf hoher See“ ersetzen. Zwei Admins schlagen verschiedene Befehle vor (rechts). Welcher erfüllt den Wunsch besser? Prüfe es erst mit SELECT.

BEFEHL 1

```
UPDATE users
SET city = 'auf hoher See'
WHERE id IN
  (SELECT user_id FROM photos
   WHERE description
     LIKE '%boot%')
```

BEFEHL 2

```
UPDATE users
SET city = 'auf hoher See'
WHERE id IN
  (SELECT photos.user_id
   FROM photos
   JOIN photo_tag ON …
   JOIN tags ON …
   WHERE tags.name = 'boot')
```

f) hugo.ranked hat sein Passwort vergessen. Setze es neu und melde dich danach als Hugo an.

g) Der Sicherheitsbeauftragte vermutet, dass manche User sehr schwache Passwörter haben. Finde mit `PASSWORD_VERIFY` heraus, wer „123456“ benutzt. Probiere auch „passwort“ und „hallo123“. Erkläre: Warum kann man den Hash nicht zurückrechnen, die Passwörter aber trotzdem finden?

* nur mit tieferen SQL-Befehlen lösbar (Abschnitt 4) · ★ zum Knobeln

6.3 Löschen

Löschen funktioniert auf die gleiche Art und Weise, nur ohne den SET-Befehl. Auch hier gilt: **Ohne WHERE wird alles gelöscht!**

AUFBAU

```
DELETE FROM <Klasse>
WHERE <Bedingung>
```

BEISPIEL · ALLE FOTOS DES USERS MIT DER ID 3 WERDEN GELÖSCHT

```
DELETE FROM photos
WHERE user_id = 3
```

NACHGEDACHT · **Was passiert mit den Kindern?**

Löschst du ein Foto, gehören die Likes und Kommentare daran zu einem Foto, das es nicht mehr gibt. In YourSozialNet sind die Fremdschlüssel deshalb mit **ON DELETE CASCADE** angelegt: Die abhängigen Zeilen werden automatisch mitgelöscht.

Aufgabe 11

Notiere für die folgenden Aufgaben einen SQL-Befehl. Prüfe vorher mit SELECT, was du löschst!

a) Lösche alle Lisas aus deiner Datenbank. Vergleiche vorher `name LIKE 'Lisa%'` und `name LIKE '%Lisa%'`. Wie viele Lisas gibt es wirklich?

b) Lösche alle Bilder von Alexander Achen. Was passiert mit den Likes und Kommentaren an seinen Bildern?

c) ★ Lösche alles, was mit Henriette Zimmerman in Verbindung steht, ihren Account jedoch nicht: alle Fotos, alle Kommentare, alle Follower, alle Abos und ihr Profilbild. *Tipp: Das Profilbild ist kein eigenes Objekt.*

d) Lösche alle Personen unter 18. Wie viele Personen waren das?

e) * Verschwörungstheorien über Echsenmenschen werden in deinem Netzwerk nicht geduldet. Lösche alle Kommentare der Echsen-Theoretiker – aber nur diese! Sofias harmlose Bartagame und die Antworten anderer User sollen bleiben. Vergleiche dazu die beiden Befehle rechts mit SELECT (vgl. Aufgabe 5 h).

BEFEHL 1

```
DELETE FROM comments
WHERE body LIKE '%Echse%'
```

BEFEHL 2

```
DELETE FROM comments
WHERE body LIKE '%Echse%'
  AND user_id IN
  (SELECT id FROM users
   WHERE username IN
   ('mondrueckseite',
    'echsenwache'))
```

f) Lösche den User aaron183. Zähle vorher und nachher alle Kommentare. Warum fehlen danach Kommentare, obwohl du in comments gar nichts gelöscht hast?

* nur mit tieferen SQL-Befehlen lösbar (Abschnitt 4) · ★ zum Knobeln

7. Analytics

YourSozialNet hat einiges über euch gespeichert – ohne dass ihr gefragt wurdet. Jedes Mal, wenn ein User das Netzwerk öffnet, beginnt eine **Session** (ein Besuch). Dabei merkt sich das Netzwerk, mit welchem Gerät, Browser und Bildschirm, von welcher IP-Adresse, in welcher Sprache und Zeitzone ihr unterwegs seid. Und jedes Foto, das ihr im Feed anschaut, wird als **View** gespeichert – mit der Zeit, die ihr darauf geblieben seid.

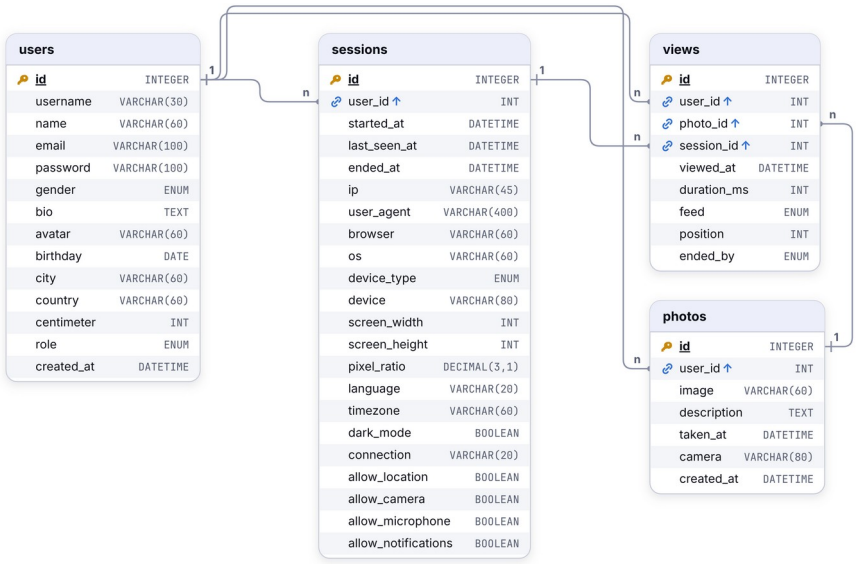


Abb. 12: sessions und views

Attribut	Bedeutung
`sessions.started_at`, `ended_at`	Beginn und Ende des Besuchs
`sessions.ip`	IP-Adresse: ungefährer Ort; gleiche IP = gleiches WLAN
`browser`, `os`, `device_type`, `device`	Browser, Betriebssystem, phone/tablet/desktop, Gerätemodell
`screen_width`, `language`, `timezone`, `dark_mode`	Bildschirm, Sprache, Zeitzone, Hell/Dunkel
`allow_location`, `allow_camera`, …	Hat der User auf „Zulassen“ (1) oder „Blockieren“ (0) getippt?
`views.duration_ms`	wie lange ein Foto angeschaut wurde (in Millisekunden)
`views.feed`, `position`, `ended_by`	in welchem Feed, an welcher Stelle, und wie es endete (weiter wischen, liken, kommentieren …)

Aufgabe 12

Findet so viel wie möglich über die User heraus – in eurem Netzwerk oder über den Kurs im Klassen-Netz. Es gibt keine feste Lösung: Jede und jeder darf etwas anderes entdecken. Nutzt dazu nicht nur sessions und views, sondern auch alle bekannten Tabellen. Notiert eure SQL-Befehle und was ihr herausgefunden habt.

- **Browser und Geräte:** Wer nutzt welchen Browser, welches Handy, welches Betriebssystem?
- **IP-Adressen:** Wer war am selben Ort? Wer hat auffällig viele verschiedene IPs?
- **Online-Zeiten:** Wann ist wer online – auch nachts?
- **Verbindungen:** WLAN oder Mobilfunk? Hell- oder Dunkelmodus? Sprache, Zeitzone?
- **Erlaubnisse:** Wer hat Standort, Kamera, Mikrofon „zugelassen“?
- **Interessen:** Welche Fotos schaut jemand am längsten an – auch ohne sie zu liken?
- **Eine Person:** Was weiß das Netzwerk alles über eine bestimmte Person?

8. Werbung

Ein richtiges soziales Netzwerk kommt nicht ohne Werbung aus – damit verdient es sein Geld. Jedoch bekommen nicht alle die gleiche Werbung. Je nach Interessen wird die Werbung möglichst **personalisiert** geschaltet. Und woher kennt das Netzwerk die Interessen? Aus genau den Daten, die du in den letzten Kapiteln abgefragt hast.

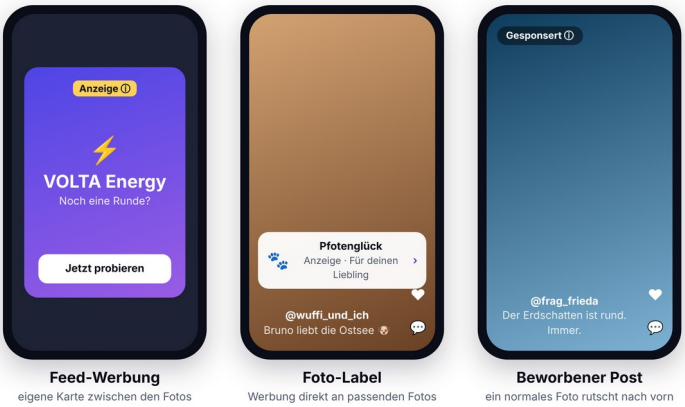


Abb. 13: Die drei Arten von Werbung in YourSozialNet

Art (kind)	Wo erscheint sie?	Was braucht sie?
Feed-Werbung (`feed`)	als eigene Karte zwischen den Fotos im Feed, z. B. nach jedem 5. Foto	Bild, Titel, Text, Knopf und **„Wem?“**
Foto-Label (`label`)	direkt an bestimmten Fotos und über deren Kommentaren	Bild, Titel, Text, **„Wem?“** und **„An welchen Fotos?“**
Beworbener Post (`boost`)	ein normales Foto rutscht im Feed „Für dich“ ganz nach vorn, mit „Gesponsert“	**„Wem?“** und **„Was?“** (welche Fotos)

Auch die Werbung ist in der Datenbank organisiert – in der Klasse **ads**:

- **id** – fortlaufende Nummer als Primärschlüssel; **user_id** ↑ – welcher Admin die Kampagne angelegt hat (NULL = die Plattform selbst)
- **name** – Name der Kampagne zum Wiederfinden; **kind** – feed, label oder boost
- **title**, **body**, **image**, **button** – was auf der Anzeige steht
- **target_sql** – **„Wem?“**: ein SQL-Befehl, der bestimmt, welche User die Werbung sehen
- **content_sql** – **„An welchen Fotos?“** (Label) bzw. **„Was?“** (Boost): ein SQL-Befehl, der eine Liste von Foto-ids liefert
- **active** – ist die Kampagne eingeschaltet?; **created_at** – wann sie angelegt wurde

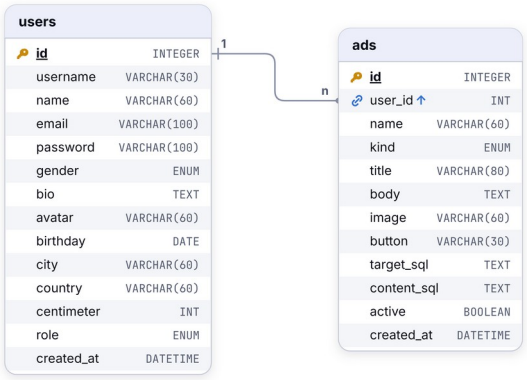


Abb. 14: Die Klasse ads

„Wem?“ – die Zielgruppe als SQL-Befehl

Der Befehl in target_sql kann zwei Formen haben. Ist er leer, sehen **alle** User die Werbung.

Form	So wird geprüft
Mit `$user`	`$user` ist ein Platzhalter für die id des Users, der gerade scrollt. Der Befehl wird für jeden User einzeln ausgeführt und liefert **einen Wert**: wahr (1) = Werbung passt, falsch (0) oder kein Ergebnis = passt nicht.

Form	So wird geprüft
Ohne `$user`	Der Befehl liefert eine **Liste von User-ids** (erste Spalte). Ist der User in der Liste, passt die Werbung.

FORM 1 (MIT $USER): WERBUNG NUR FÜR USERINNEN – „GLOW SKIN“

```
SELECT CASE gender
         WHEN 'female' THEN true
         ELSE false
       END
FROM users
WHERE id = $user
```

`CASE … WHEN … THEN … ELSE … END` funktioniert wie ein if-else in Java: Ist gender gleich „female“, ist das Ergebnis true, sonst false.

FORM 2 (LISTE): ALLE USER, DIE MINDESTENS 3 GAMING-FOTOS GELIKT HABEN – „VOLTA ENERGY“

```
SELECT likes.user_id
FROM likes
JOIN photo_tag ON photo_tag.photo_id = likes.photo_id
JOIN tags      ON tags.id = photo_tag.tag_id
WHERE tags.name IN ('gaming', 'setup', 'stream')
GROUP BY likes.user_id
HAVING COUNT(*) >= 3
```

„AN WELCHEN FOTOS?“: ALLE FOTOS VON HAUSTIEREN – „PFOTENGLÜCK“

```
SELECT photo_tag.photo_id
FROM photo_tag
JOIN tags ON tags.id = photo_tag.tag_id
WHERE tags.name IN ('hund', 'katze', 'hamster', 'haustier')
```

Aufgabe 13

Neben dem Betreiben deines unglaublich erfolgreichen sozialen Netzwerks leitest du auch die Werbeabteilung.

a) Schau dir die Kampagnen in deinem Netzwerk an (`SELECT * FROM ads` oder Reiter **Werbung**). Wer sieht sie und warum? Welche findest du problematisch?

b) Denkt euch einen Werbekunden aus (Produkt, Firma, Partei oder Organisation) und überlegt, welche User-Gruppen ihr wie ansprechen wollt.

c) Entwickelt **drei Werbungen** für euren Kunden – eine **Feed-Werbung**, ein **Foto-Label** und einen **beworbenen Post**, jede mit eigener Zielgruppe per SQL. Testet sie in eurem Netzwerk (Probelauf, Vorschau) und schaltet sie dann im **Klassen-Netz**: Möglichst jede Person im Kurs soll eine passende Werbung bekommen.